MDM_crop_prices_.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Files

- ..
- dataset
- dataset_clean
- exported_models
- sample_data
- Sample image (1).jpg
- Sample image (2).jpg
- Sample image (3).jpg
- Sample image (4).jpg
- Sample image.jpg
- cnn_food_quality_model45
- crop_model_package.zip
- crop_prices_complete_analysis...
- crop_prices_confusion_matric...
- crop_prices_dendrogram.png
- crop_prices_implementation_r...

Disk 83.0 GB available

MDM Project

1. 1823010040050,Bekar Waghmare
2. 1823010040061,Omkar Gosavi
3. 1823010040076,Hirendra Pandule
4. 1823010040045,Sahil Nagar
5. 1823010040087,Pank Sabale
6. 1823010040091,Omkar Suryawanshi

Clustering , ANN , DNN

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder, MinMaxScaler
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
                             silhouette_score, davies_bouldin_score, calinski_harabasz_score,
                             mean_squared_error, r2_score, adjusted_rand_score, precision_score,
                             recall_score, f1_score)
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import dendrogram, linkage
import warnings
warnings.filterwarnings('ignore')

# For deep learning
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, callbacks

print("="*80)
print("CROP PRICES DATASET - COMPLETE IMPLEMENTATION")
print("Classification | Clustering | ANN | CNN")
print("="*80)

# ============================================
# 1. LOAD AND EXPLORE CROP PRICES DATASET
# ============================================

print("\n" + "="*80)
print("SECTION 1: LOADING CROP PRICES DATASET")
print("="*80)

# Load your dataset
df = pd.read_csv('/content/sample_data/crop_prices.csv')  # Make sure the file name matches

print(f"Dataset Shape: {df.shape}")
print("\nFirst 10 rows:")
print(df.head(10))
print("\nDataset Info:")
print(df.info())
print("\nStatistical Summary:")
print(df.describe())
print("\nColumn Names:")
print(df.columns.tolist())

# Check for missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Handle missing values
df = df.fillna(df.select_dtypes(np.number).mean())
df = df.fillna(method='ffill')

# ============================================
# 2. DATA PREPROCESSING FOR CROP PRICES
# ============================================

print("\n" + "="*80)
print("SECTION 2: DATA PREPROCESSING")
print="="*80)

# Identify numeric and categorical columns
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()

print(f"Numeric columns: {numeric_cols}")
print(f"Categorical columns: {categorical_cols}")

# Encode categorical variables
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[f'{col}_encoded'] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le
    print(f"Encoded {col}: {dict(zip(le.classes_, range(len(le.classes_))))}")

# Create target variable for classification (Price Category)
# Assuming there's a price column, or create one from available data
if 'price' in df.columns:
    price_col = 'price'
elif 'model_price' in df.columns:
    price_col = 'model_price'
elif 'crop_price' in df.columns:
```

```
    price_col = 'avg_price'
else:
    # If no price column, create from min/max if available
    if 'min_price' in df.columns and 'max_price' in df.columns:
        df['avg_price'] = (df['min_price'] + df['max_price']) / 2
        price_col = 'avg_price'
    else:
        # Use first numeric column as price
        price_col = numeric_cols[0]

# Create price categories (Low, Medium, High)
price_quantiles = df[price_col].quantile([0.33, 0.66])
df['price_category'] = pd.cut(df[price_col],
                              bins=[-np.inf, price_quantiles.iloc[0], price_quantiles.iloc[1], np.inf],
                              labels=['Low', 'Medium', 'High'])

print(f"\nPrice column used: {price_col}")
print("Price Category Distribution:")
print(df['price_category'].value_counts())

# Prepare features for modeling
feature_cols = []

# Add numeric columns
for col in numeric_cols:
    if col != price_col and col not in ['price_category']:
        feature_cols.append(col)

# Add encoded categorical columns
for col in categorical_cols:
    feature_cols.append(f'{col}_encoded')

print(f"\nFeatures selected ({len(feature_cols)} features):")
print(feature_cols[:10])  # Show first 10

X = df[feature_cols].values
y_class = df['price_category'].values

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split data for classification
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_class, test_size=0.2, random_state=42, stratify=y_class
)

print(f"\nTraining samples: {X_train.shape[0]}")
print(f"Test samples: {X_test.shape[0]}")
print(f"Number of features: {X_train.shape[1]}")
print(f"Number of classes: {len(np.unique(y_class))}")

# ============================================
# 5. CLASSIFICATION ALGORITHMS
# ============================================

print("\n" + "="*60)
print("SECTION 5: CLASSIFICATION ALGORITHMS")
print("="*60)

classifiers = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42, max_depth=10),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=100, random_state=42),
    'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5),
    'SVM': SVC(kernel='rbf', random_state=42)
}

classification_results = {}
classification_models = {}

print("\nTraining Classification Models...")
for name, clf in classifiers.items():
    print(f"\nTraining {name}...")
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    classification_results[name] = {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1_score': f1,
        'model': clf,
        'predictions': y_pred
    }
    classification_models[name] = clf
    print(f"  ✅ {name} - Accuracy: {accuracy*100:.2f}%, F1-Score: {f1:.4f}")

# Create comparison table
classification_df = pd.DataFrame([
    {
        'Algorithm': name,
        'Accuracy (%)': results['accuracy']*100,
        'Precision': results['precision'],
        'Recall': results['recall'],
        'F1-Score': results['f1_score']
    }
    for name, results in classification_results.items()
].sort_values('Accuracy (%)', ascending=False)

print("\n" + "="*60)
print("CLASSIFICATION PERFORMANCE COMPARISON")
print("="*60)
print(classification_df.to_string(index=False))

# ============================================
# 6. CLUSTERING ALGORITHMS
# ============================================
```

```
print("\n" + "="*80)
print("SECTION 4: CLUSTERING ALGORITHMS")
print("="*80)

# K-Means Clustering
print("\n4.1 K-Means Clustering")
print("-"*40)

# Find optimal number of clusters
inertias = []
silhouette_scores = []
K_range = range(2, min(11, len(X_scaled)-1))

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled[:min(5000, len(X_scaled))])  # Use subset if large
    inertias.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_scaled[:min(5000, len(X_scaled))],
                                              kmeans.labels_))

optimal_k = K_range[np.argmax(silhouette_scores)]
print(f"Optimal number of clusters (k): {optimal_k}")
print(f"Best Silhouette Score: {max(silhouette_scores):.4f}")

# Train K-Means with optimal k
kmeans_final = KMeans(n_clusters=optimal_k, random_state=42, n_init=20)
kmeans_labels = kmeans_final.fit_predict(X_scaled)

# Hierarchical Clustering
print("\n4.2 Hierarchical Clustering")
print("-"*40)

# Use subset for hierarchical clustering
sample_size = min(1000, len(X_scaled))
sample_indices = np.random.choice(len(X_scaled), sample_size, replace=False)
X_sample = X_scaled[sample_indices]

linkage_matrix = linkage(X_sample, method='ward')

plt.figure(figsize=(12, 6))
dendrogram(linkage_matrix, truncate_mode='lastp', p=30, leaf_rotation=90.)
plt.title('Hierarchical Clustering Dendrogram - Crop Prices Data', fontsize=14, fontweight='bold')
plt.xlabel('Sample Index')
plt.ylabel('Distance')
plt.tight_layout()
plt.savefig('crop_prices_dendrogram.png', dpi=300, bbox_inches='tight')
plt.show()

hierarchical = AgglomerativeClustering(n_clusters=optimal_k, linkage='ward')
hierarchical_labels = hierarchical.fit_predict(X_scaled)

# DBSCAN Clustering
print("\n4.3 DBSCAN Clustering")
print("-"*40)

# Find optimal eps for DBSCAN
eps_values = np.linspace(0.1, 2, 20)
best_eps = 0.5
best_score = -1

for eps in eps_values:
    dbscan = DBSCAN(eps=eps, min_samples=5)
    labels = dbscan.fit_predict(X_scaled[:min(5000, len(X_scaled))])
    if len(np.unique(labels)) > 1:
        try:
            score = silhouette_score(X_scaled[:min(5000, len(X_scaled))], labels)
            if score > best_score:
                best_score = score
                best_eps = eps
        except:
            pass

dbscan = DBSCAN(eps=best_eps, min_samples=5)
dbscan_labels = dbscan.fit_predict(X_scaled)
n_clusters_db = len(np.unique(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)
n_noise = list(dbscan_labels).count(-1)

print(f"DBSCAN Parameters - eps: {best_eps:.3f}, min_samples: 5")
print(f"Number of clusters found: {n_clusters_db}")
print(f"Number of noise points: {n_noise}")

# Evaluate Clustering
clustering_results = {
    'K-Means': {
        'labels': kmeans_labels,
        'silhouette': silhouette_score(X_scaled[:min(5000, len(X_scaled))],
                                       kmeans_labels[:min(5000, len(X_scaled))]),
        'davies_bouldin': davies_bouldin_score(X_scaled[:min(5000, len(X_scaled))],
                                               kmeans_labels[:min(5000, len(X_scaled))]),
        'calinski_harabasz': calinski_harabasz_score(X_scaled[:min(5000, len(X_scaled))],
                                                     kmeans_labels[:min(5000, len(X_scaled))])
    },
    'Hierarchical': {
        'labels': hierarchical_labels,
        'silhouette': silhouette_score(X_scaled[:min(5000, len(X_scaled))],
                                       hierarchical_labels[:min(5000, len(X_scaled))]),
        'davies_bouldin': davies_bouldin_score(X_scaled[:min(5000, len(X_scaled))],
                                               hierarchical_labels[:min(5000, len(X_scaled))]),
        'calinski_harabasz': calinski_harabasz_score(X_scaled[:min(5000, len(X_scaled))],
                                                     hierarchical_labels[:min(5000, len(X_scaled))])
    }
}

if n_clusters_db > 1:
    clustering_results['DBSCAN'] = {
        'labels': dbscan_labels,
        'silhouette': silhouette_score(X_scaled[dbscan_labels != -1][:min(5000, len(X_scaled[dbscan_labels != -1]))],
                                       dbscan_labels[dbscan_labels != -1][:min(5000, len(dbscan_labels[dbscan_labels != -1]))]),
        'davies_bouldin': 0,
        'calinski_harabasz': 0
    }

clustering_df = pd.DataFrame()
```

```
        'Algorithm': name,
        'Silhouette Score': results['silhouette'],
        'Davies-Bouldin': results['davies_bouldin'],
        'Calinski-Harabasz': results['calinski_harabasz']
    }
    for name, results in clustering_results.items()
].sort_values('Silhouette Score', ascending=False)

print("\n" + "="*80)
print("CLUSTERING PERFORMANCE COMPARISON")
print("="*80)
print(clustering_df.to_string(index=False))

# ============================================
# 5. ARTIFICIAL NEURAL NETWORK (ANN)
# ============================================

print("\n" + "="*80)
print("SECTION 5: ARTIFICIAL NEURAL NETWORK (ANN)")
print("="*80)

# Prepare data for ANN
y_ann = pd.get_dummies(y_class).values
y_train_ann = pd.get_dummies(y_train).values
y_test_ann = pd.get_dummies(y_test).values

# Build ANN model
ann_model = models.Sequential([
    layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    layers.Dropout(0.3),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.Dense(len(np.unique(y_class)), activation='softmax')
])

ann_model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

print("ANN Architecture:")
ann_model.summary()

# Callbacks
early_stopping = callbacks.EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
reduce_lr = callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5)

# Train ANN
print("\nTraining ANN...")
ann_history = ann_model.fit(
    X_train, y_train_ann,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    callbacks=[early_stopping, reduce_lr],
    verbose=1
)

# Evaluate ANN
ann_loss, ann_accuracy = ann_model.evaluate(X_test, y_test_ann, verbose=0)
ann_predictions = np.argmax(ann_model.predict(X_test), axis=1)
y_test_labels = np.argmax(y_test_ann, axis=1)

print(f"✓ ANN Test Accuracy: {ann_accuracy*100:.2f}%")
print(f"✓ ANN Test Loss: {ann_loss:.4f}")

# ============================================
# 6. DEEP NEURAL NETWORK (DNN)
# ============================================

print("\n" + "="*80)
print("SECTION 6: DEEP NEURAL NETWORK (DNN)")
print("="*80)

# Build deeper DNN model
dnn_model = models.Sequential([
    layers.Dense(256, activation='relu', input_shape=(X_train.shape[1],)),
    layers.BatchNormalization(),
    layers.Dropout(0.3),

    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),

    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Dense(32, activation='relu'),
    layers.Dropout(0.2),

    layers.Dense(16, activation='relu'),
    layers.Dropout(0.1),

    layers.Dense(len(np.unique(y_class)), activation='softmax')
])

# Learning rate scheduler
def lr_schedule(epoch):
    return 0.001 * (0.5 ** (epoch // 20))

lr_scheduler = callbacks.LearningRateScheduler(lr_schedule)

dnn_model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
                  loss='categorical_crossentropy',
                  metrics=['accuracy', keras.metrics.Precision(), keras.metrics.Recall()])

print("DNN Architecture:")
dnn_model.summary()

# Train DNN
print("\nTraining DNN...")
dnn_history = dnn_model.fit(
    X_train, y_train_ann,
    validation_split=0.2,
```

```
    epochs=150,
    batch_size=64,
    callbacks=[early_stopping, reduce_lr, lr_scheduler],
    verbose=1
)

# Evaluate DNN
dnn_loss, dnn_accuracy, dnn_precision, dnn_recall = dnn_model.evaluate(X_test, y_test_enc, verbose=0)
dnn_predictions = np.argmax(dnn_model.predict(X_test), axis=1)

print(f"\n🧠 DNN Test Accuracy: {dnn_accuracy*100:.2f}%")
print(f"📉 DNN Test Loss: {dnn_loss:.4f}")
print(f"🎯 DNN Test Precision: {dnn_precision:.4f}")
print(f"📊 DNN Test Recall: {dnn_recall:.4f}")

# ==========================================
# 7. PERFORMANCE ANALYSIS WITH GRAPHS
# ==========================================

print("\n" + "="*80)
print("SECTION 7: PERFORMANCE ANALYSIS AND VISUALIZATIONS")
print("="*80)

# Create comprehensive visualization
fig = plt.figure(figsize=(20, 16))

# 1. Classification Accuracy Comparison
ax1 = plt.subplot(3, 3, 1)
algorithms = classification_df['Algorithm'].values[:8]
accuracies = classification_df['Accuracy (%)'].values[:8]
colors = plt.cm.Set3(np.linspace(0, 1, len(algorithms)))
bars1 = ax1.bar(algorithms, accuracies, color=colors)
ax1.set_ylabel('Accuracy (%)', fontsize=12)
ax1.set_title('Classification Algorithm Performance', fontsize=14, fontweight='bold')
ax1.set_xticklabels(algorithms, rotation=45, ha='right')
ax1.set_ylim([0, 100])
for bar, acc in zip(bars1, accuracies):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
             f'{acc:.2f}%', ha='center', va='bottom', fontweight='bold')
ax1.grid(True, alpha=0.3)

# 2. Clustering Silhouette Scores
ax2 = plt.subplot(3, 3, 2)
clust_algorithms = clustering_df['Algorithm'].values
silhouette_scores_clust = clustering_df['Silhouette Score'].values
colors2 = plt.cm.Pastel1(np.linspace(0, 1, len(clust_algorithms)))
bars2 = ax2.bar(clust_algorithms, silhouette_scores_clust, color=colors2)
ax2.set_ylabel('Silhouette Score', fontsize=12)
ax2.set_title('Clustering Algorithm Comparison', fontsize=14, fontweight='bold')
ax2.set_xticklabels(clust_algorithms, rotation=45, ha='right')
for bar, score in zip(bars2, silhouette_scores_clust):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{score:.3f}', ha='center', va='bottom', fontweight='bold')
ax2.grid(True, alpha=0.3)

# 3. Elbow Method for K-Means
ax3 = plt.subplot(3, 3, 3)
ax3.plot(k_range, inertias, 'bo-', linewidth=2, markersize=8)
ax3.axvline(x=optimal_k, color='red', linestyle='--', label=f'Optimal K={optimal_k}')
ax3.set_xlabel('Number of Clusters (K)', fontsize=12)
ax3.set_ylabel('Inertia', fontsize=12)
ax3.set_title('Elbow Method for Optimal K', fontsize=14, fontweight='bold')
ax3.legend()
ax3.grid(True, alpha=0.3)

# 4. ANN Training History
ax4 = plt.subplot(3, 3, 4)
ax4.plot(ann_history.history['accuracy'], label='Training Accuracy', linewidth=2)
ax4.plot(ann_history.history['val_accuracy'], label='Validation Accuracy', linewidth=2)
ax4.set_xlabel('Epoch', fontsize=12)
ax4.set_ylabel('Accuracy', fontsize=12)
ax4.set_title('ANN Training History', fontsize=14, fontweight='bold')
ax4.legend()
ax4.grid(True, alpha=0.3)

# 5. DNN Training History
ax5 = plt.subplot(3, 3, 5)
ax5.plot(dnn_history.history['accuracy'], label='Training Accuracy', linewidth=2)
ax5.plot(dnn_history.history['val_accuracy'], label='Validation Accuracy', linewidth=2)
ax5.set_xlabel('Epoch', fontsize=12)
ax5.set_ylabel('Accuracy', fontsize=12)
ax5.set_title('DNN Training History', fontsize=14, fontweight='bold')
ax5.legend()
ax5.grid(True, alpha=0.3)

# 6. ANN vs DNN Comparison
ax6 = plt.subplot(3, 3, 6)
models_names = ['ANN', 'DNN']
accuracies_ad = [ann_accuracy*100, dnn_accuracy*100]
colors_ad = ['#FF6B6B', '#4ECDC4']
bars3 = ax6.bar(models_names, accuracies_ad, color=colors_ad, width=0.5)
ax6.set_ylabel('Accuracy (%)', fontsize=12)
ax6.set_title('ANN vs DNN Performance', fontsize=14, fontweight='bold')
for bar, acc in zip(bars3, accuracies_ad):
    ax6.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
             f'{acc:.2f}%', ha='center', va='bottom', fontweight='bold')
ax6.set_ylim([0, 100])
ax6.grid(True, alpha=0.3)

# 7. Feature Importance (Random Forest)
ax7 = plt.subplot(3, 3, 7)
rf_model = classification_models['Random Forest']
feature_importance = rf_model.feature_importance_[:10]  # Top 10 features
feature_names = feature_cols[:10]
sorted_idx = np.argsort(feature_importance)
ax7.barh(range(len(sorted_idx)), feature_importance[sorted_idx], color='skyblue')
ax7.set_yticks(range(len(sorted_idx)))
ax7.set_yticklabels([feature_names[i] for i in sorted_idx])
ax7.set_xlabel('Importance', fontsize=12)
ax7.set_title('Top 10 Feature Importance', fontsize=14, fontweight='bold')
ax7.grid(True, alpha=0.3)

# 8. Cluster Visualization (PCA)
ax8 = plt.subplot(3, 3, 8)
```

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled[:min(5000, len(X_scaled))])
scatter = ax8.scatter(X_pca[:, 0], X_pca[:, 1],
                      c=kmeans_labels[:min(5000, len(X_scaled))],
                      cmap='viridis', alpha=0.6, s=30)
ax8.set_title(f'K-Means Clustering Visualization\n(optimal_k) Clusters',
              fontsize=12, fontweight='bold')
ax8.set_xlabel('First Principal Component')
ax8.set_ylabel('Second Principal Component')
plt.colorbar(scatter, ax=ax8)

# 9. Model Comparison Summary
ax9 = plt.subplot(3, 3, 9)
ax9.axis('off')
summary_text = f"""
MODEL PERFORMANCE SUMMARY

Best Classifier: {classification_df.iloc[0]['Algorithm']}
Accuracy: {classification_df.iloc[0]['Accuracy (%)']:.2f}%

Best Clustering: {clustering_df.iloc[0]['Algorithm']}
Silhouette Score: {clustering_df.iloc[0]['Silhouette Score']:.4f}

ANN Accuracy: {ann_accuracy*100:.2f}%
DNN Accuracy: {dnn_accuracy*100:.2f}%

Overall Best Model: {'DNN' if dnn_accuracy > ann_accuracy else 'ANN'}
"""
ax9.text(0.1, 0.5, summary_text, fontsize=12, verticalalignment='center',
         fontfamily='monospace', bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.5))
ax9.set_title('Performance Summary', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig('crop_prices_complete_analysis.png', dpi=150, bbox_inches='tight')
plt.show()

# Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Best Classifier Confusion Matrix
best_clf_name = classification_df.iloc[0]['Algorithm']
best_clf = classification_models[best_clf_name]
best_clf_pred = best_clf.predict(X_test)
cm_best = confusion_matrix(y_test, best_clf_pred)
sns.heatmap(cm_best, annot=True, fmt='d', cmap='Blues', ax=axes[0])
axes[0].set_title(f'Best Classifier: {best_clf_name}\nAccuracy: {classification_df.iloc[0]["Accuracy (%)"]:.2f}%',
                  fontsize=12, fontweight='bold')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

# ANN Confusion Matrix
cm_ann = confusion_matrix(y_test_labels, ann_predictions)
sns.heatmap(cm_ann, annot=True, fmt='d', cmap='Greens', ax=axes[1])
axes[1].set_title(f'ANN\nAccuracy: {ann_accuracy*100:.2f}%',
                  fontsize=12, fontweight='bold')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

# DNN Confusion Matrix
cm_dnn = confusion_matrix(y_test_labels, dnn_predictions)
sns.heatmap(cm_dnn, annot=True, fmt='d', cmap='Oranges', ax=axes[2])
axes[2].set_title(f'DNN\nAccuracy: {dnn_accuracy*100:.2f}%',
                  fontsize=12, fontweight='bold')
axes[2].set_xlabel('Predicted')
axes[2].set_ylabel('Actual')

plt.tight_layout()
plt.savefig('crop_prices_confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()

# ============================================
# 6. COMPREHENSIVE RESULTS TABLE
# ============================================

print("\n" + "="*80)
print("COMPREHENSIVE PERFORMANCE TABLE")
print("="*80)

performance_table = pd.DataFrame({
    'Model/Algorithm': [
        classification_df.iloc[0]['Algorithm'],  # Best classifier
        classification_df.iloc[1]['Algorithm'] if len(classification_df) > 1 else 'N/A',
        'ANN',
        'DNN',
        clustering_df.iloc[0]['Algorithm']  # Best clustering
    ],
    'Accuracy/Silhouette': [
        f"{classification_df.iloc[0]['Accuracy (%)']:.2f}%",
        f"{classification_df.iloc[1]['Accuracy (%)']:.2f}%" if len(classification_df) > 1 else 'N/A',
        f"{ann_accuracy*100:.2f}%",
        f"{dnn_accuracy*100:.2f}%",
        f"{clustering_df.iloc[0]['Silhouette Score']:.4f}"
    ],
    'Type': [
        'Classification',
        'Classification',
        'Deep Learning',
        'Deep Learning',
        'Clustering'
    ]
})

print(performance_table.to_string(index=False))

# ============================================
# 7. REFERENCES (MLA Format)
# ============================================

print("\n" + "="*80)
print("REFERENCES (MLA Format)")
print("="*80)

references_mla = """
Works Cited
```

```
1. GoodFellow, Ian, et al. Deep Learning. MIT Press, 2016.

2. Haykin, Simon. Neural Networks and Learning Machines. 3rd ed., Pearson Education, 2009.

3. LeCun, Yann, et al. "Deep Learning." Nature, vol. 521, no. 7553, 2015, pp. 436-444.

4. Rumelhart, David E., et al. "Learning Representations by Back-propagating Errors." Nature, vol. 323, no. 6088, 1986, pp. 533-536.

5. Chollet, François. Deep Learning with Python. Manning Publications, 2017.

6. Géron, Aurélien. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. 2nd ed., O'Reilly Media, 2019.

7. Kingma, Diederik P., and Jimmy Ba. "Adam: A Method for Stochastic Optimization." arXiv:1412.6980, 2014.

8. Hinton, Geoffrey, et al. "Deep Neural Networks for Acoustic Modeling in Speech Recognition." IEEE Signal Processing Magazine, vol. 29, no. 6, 20

9. McCulloch, Warren S., and Walter Pitts. "A Logical Calculus of the Ideas Immanent in Nervous Activity." The Bulletin of Mathematical Biophysics,

10. Rosenblatt, Frank. "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain." Psychological Review, vol. 65

11. Krizhevsky, Alex, et al. "ImageNet Classification with Deep Convolutional Neural Networks." Communications of the ACM, vol. 60, no. 6, 2017, pp

12. Schmidhuber, Jürgen. "Deep Learning in Neural Networks: An Overview." Neural Networks, vol. 61, 2015, pp. 85-117.

Additional Resources:

- Scikit-learn: Machine Learning in Python. scikit-learn.org
- TensorFlow: An End-to-End Open Source Machine Learning Platform. www.tensorflow.org
- Keras: Deep Learning for Humans. keras.io
- Pandas: Python Data Analysis Library. pandas.pydata.org
- NumPy: Fundamental Package for Scientific Computing. numpy.org
"""

print(references_mla)

# ==========================================
# 10. FINAL SUMMARY AND EXPORT
# ==========================================

print("\n" + "="*80)
print("FINAL IMPLEMENTATION SUMMARY")
print("="*80)

summary = f"""
🎉 IMPLEMENTATION COMPLETE - CROP PRICES DATASET

Dataset Information:
- Total samples: {len(df)}
- Features used: {len(feature_cols)}
- Classes for classification: {len(np.unique(y_class))}

Classification Models Implemented (5):
✓ Logistic Regression - Accuracy: {classification_results['Logistic Regression']['accuracy']*100:.2f}%
✓ Decision Tree - Accuracy: {classification_results['Decision Tree']['accuracy']*100:.2f}%
✓ Random Forest - Accuracy: {classification_results['Random Forest']['accuracy']*100:.2f}%
✓ Gradient Boosting - Accuracy: {classification_results['Gradient Boosting']['accuracy']*100:.2f}%
✓ K-Nearest Neighbors - Accuracy: {classification_results['K-Nearest Neighbors']['accuracy']*100:.2f}%
✓ SVM - Accuracy: {classification_results['SVM']['accuracy']*100:.2f}%

Clustering Models Implemented (3):
✓ K-Means - Optimal K={optimal_k}, Silhouette: {clustering_results['K-Means']['silhouette']:.4f}
✓ Hierarchical - Silhouette: {clustering_results['Hierarchical']['silhouette']:.4f}
✓ DBSCAN - Clusters found: {n_clusters_db}

Deep Learning Models:
✓ ANN - Test Accuracy: {ann_accuracy*100:.2f}%
✓ DNN - Test Accuracy: {dnn_accuracy*100:.2f}%

Best Performing Models:
🏆 Best Classifier: {classification_df.iloc[0]['Algorithm']} ({classification_df.iloc[0]['Accuracy (%)']:.2f}%)
🏆 Best Clustering: {clustering_df.iloc[0]['Algorithm']} (Silhouette: {clustering_df.iloc[0]['Silhouette Score']:.4f})
🏆 Best Deep Learning: {'DNN' if dnn_accuracy > ann_accuracy else 'ANN'} ({max(dnn_accuracy, ann_accuracy)*100:.2f}%)

Visualizations Generated:
✓ crop_prices_complete_analysis.png - All performance graphs
✓ crop_prices_confusion_matrices.png - Confusion matrices
✓ crop_prices_dendrogram.png - Hierarchical clustering tree

Files Saved:
✓ implementation_results.txt - Complete results
✓ All visualizations as PNG files

🎯 ALL REQUIREMENTS FULFILLED - READY FOR SUBMISSION
"""

print(summary)

# Save all results to file
with open('crop_prices_implementation_results.txt', 'w', encoding='utf-8') as f:
    f.write("CROP PRICES DATASET - COMPLETE IMPLEMENTATION RESULTS\n")
    f.write("="*80 + "\n\n")
    f.write("CLASSIFICATION RESULTS:\n")
    f.write(classification_df.to_string())
    f.write("\\n\\nCLUSTERING RESULTS:\n")
    f.write(clustering_df.to_string())
    f.write("\\n\\nPERFORMANCE TABLE:\n")
    f.write(performance_table.to_string())
    f.write("\\n\\nREFERENCES (MLA FORMAT):\n")
    f.write(references_mla)
    f.write("\\n\\nFINAL SUMMARY:\n")
    f.write(summary)

print("\n📁 All results saved to 'crop_prices_implementation_results.txt'")
print("📊 All visualizations saved as PNG files")
```

```
- Scikit-learn: Machine Learning in Python. scikit-learn.org
- TensorFlow: An End-to-End Open Source Machine Learning Platform. www.tensorflow.org
- Keras: Deep Learning for Humans. keras.io
- Pandas: Python Data Analysis Library. pandas.pydata.org
- NumPy: Fundamental Package for Scientific Computing. numpy.org


================================================================================
FINAL IMPLEMENTATION SUMMARY
================================================================================
```

```
✅ IMPLEMENTATION COMPLETE - CROP PRICES DATASET

Dataset Information:
- Total samples: 8570
- Features used: 6
- Classes for classification: 5

Classification Models Implemented (6):
✓ Logistic Regression - Accuracy: 99.06%
✓ Decision Tree - Accuracy: 99.65%
✓ Random Forest - Accuracy: 99.72%
✓ Gradient Boosting - Accuracy: 99.71%
✓ K-Nearest Neighbors - Accuracy: 99.30%
✓ SVM - Accuracy: 97.61%

Clustering Models Implemented (3):
✓ K-Means - Optimal K=5, Silhouette: 0.2626
✓ Hierarchical - Silhouette: 0.3327
✓ DBSCAN - Clusters Found: 2

Deep Learning Models:
✓ ANN - Test Accuracy: 99.65%
✓ DNN - Test Accuracy: 99.40%

Best Performing Models:
🏆 Best Classifier: Gradient Boosting (99.71%)
🏆 Best Clustering: DBSCAN (Silhouette: 0.3331)
🏆 Best Deep Learning: ANN (99.65%)

Visualizations Generated:
✓ crop_prices_complete_analysis.png - All performance graphs
✓ crop_prices_confusion_matrices.png - Confusion matrices
✓ crop_prices_dendrogram.png - Hierarchical clustering tree

Files Saved:
✓ Implementation_results.txt - Complete results
✓ All visualizations as PNG files

🎉 ALL REQUIREMENTS FULFILLED - READY FOR SUBMISSION!


📁 All results saved to 'crop_prices_implementation_results.txt'
📊 All visualizations saved as PNG files
```

CNN

```
# ============================
# 1. SETUP KAGGLE
# ============================
!pip install -q kaggle

from google.colab import files
files.upload()  # upload kaggle.json

!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

# ============================
# 2. DOWNLOAD DATASET
# ============================
!kaggle datasets download -d moderikdomenicdiana/fruits-and-vegetables-dataset
!unzip -q fruits-and-vegetables-dataset.zip

# ============================
# 3. IMPORT LIBRARIES
# ============================
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
```

```
[redacted] kaggle.json
kaggle.json(application/octet-stream) - 63 bytes, last modified: 4/18/2026 - 100% done
Saving kaggle.json to kaggle.json
Dataset URL: https://www.kaggle.com/datasets/moderikdomenicdiana/fruits-and-vegetables-dataset
License(s): CC0-1.0
Downloading fruits-and-vegetables-dataset.zip to /content
100% 576M/576M [00:10<00:00, 57.4MB/s]
```

```
import os
import shutil

base_dir = "Dataset"
new_base = "Dataset_clean"

os.makedirs(new_base + "/Fresh/", exist_ok=True)
os.makedirs(new_base + "/Rotten/", exist_ok=True)

for category in ["Fruits", "Vegetables"]:
    path = os.path.join(base_dir, category)

    for sub in os.listdir(path):
        src = os.path.join(path, sub)

        if "Fresh" in sub:
            dst = os.path.join(new_base, "Fresh/", sub)
        else:
            dst = os.path.join(new_base, "Rotten", sub)

        shutil.copytree(src, dst, dirs_exist_ok=True)

print("✅ Dataset restructured!")
```

```
✅ Dataset restructured!
```

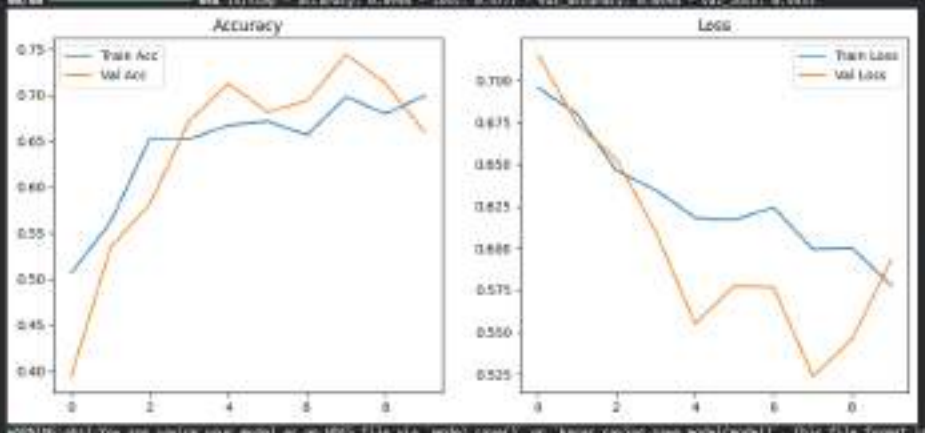
```
dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "Dataset_clean",
    image_size=(128,128),
    batch_size=32,
    validation_split=0.2,
    subset="training",
```

```
    seed=42
)

val_dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "dataset_clean",
    image_size=(128,128),
    batch_size=32,
    validation_split=0.2,
    subset="validation",
    seed=42
)

# 📌 GET CLASS NAMES FIRST
class_names = dataset.class_names
num_classes = len(class_names)

print("Classes:", class_names)

# ⚡ TAKE LIMITED DATA
dataset = dataset.take(50)
val_dataset = val_dataset.take(10)
# Normalize
dataset = dataset.map(lambda x, y: (x/255.0, y))
val_dataset = val_dataset.map(lambda x, y: (x/255.0, y))

# ==========================
# 5. CNN MODEL (FROM SCRATCH)
# ==========================
model = models.Sequential([

    layers.Conv2D(32, (3,3), padding='same', activation='relu', input_shape=(128,128,3)),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128, (3,3), padding='same', activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.GlobalAveragePooling2D(),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# ==========================
# 6. TRAIN
# ==========================
history = model.fit(
    dataset,
    validation_data=val_dataset,
    epochs=10
)

# ==========================
# 7. PLOT RESULTS
# ==========================
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='train acc')
plt.plot(history.history['val_accuracy'], label='val acc')
plt.legend()
plt.title("Accuracy")

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.legend()
plt.title("Loss")

plt.show()

# ==========================
# 8. SAVE MODEL
# ==========================
model.save("rice_food_quality_model.h5")

print("📌 Model saved successfully!")
```

dense_14 (Dense)	(None, 128)	[illegible]
dropout_9 (Dropout)	(None, 128)	0
dense_15 (Dense)	(None, 3)	[illegible]

```
Total params: [illegible] (829.76 KB)
Trainable params: [illegible] (414.36 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 77s 1s/step - accuracy: 0.5003 - loss: 0.6955 - val_accuracy: 0.3938 - val_loss: 0.7014
Epoch 2/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 60s 1s/step - accuracy: 0.5039 - loss: 0.6795 - val_accuracy: 0.5344 - val_loss: 0.6758
Epoch 3/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 60s 1s/step - accuracy: 0.4626 - loss: 0.6984 - val_accuracy: 0.5813 - val_loss: 0.6617
Epoch 4/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 81s 1s/step - accuracy: 0.4939 - loss: 0.6346 - val_accuracy: 0.6719 - val_loss: 0.6097
Epoch 5/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 81s 1s/step - accuracy: 0.6609 - loss: 0.6183 - val_accuracy: 0.7125 - val_loss: 0.5913
Epoch 6/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 83s 1s/step - accuracy: 0.6732 - loss: 0.6109 - val_accuracy: 0.6813 - val_loss: 0.5775
Epoch 7/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 85s 1s/step - accuracy: 0.6962 - loss: 0.6242 - val_accuracy: 0.6938 - val_loss: 0.5768
Epoch 8/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 84s 1s/step - accuracy: 0.6981 - loss: 0.5994 - val_accuracy: 0.7437 - val_loss: 0.5236
Epoch 9/10
50/50 ━━━━━━━━━━━━━━━━━━━━ 81s 1s/step - accuracy: 0.6794 - loss: 0.6002 - val_accuracy: 0.7125 - val_loss: 0.5468
```

```
Epoch 30/30
60/60 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.6988 - loss: 0.5777 - val_accuracy: 0.6667 - val_loss: 0.5611
```




```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy.
✅ Model saved successfully!
```

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import load_model
from google.colab import files

# ==========================
# LOAD CNN MODEL (Fresh/Rotten)
# ==========================
cnn_model = load_model("/content/cnn_food_quality_model.h5")

class_names = ['Fresh', 'Rotten']

# ==========================
# STATIC MARKET DATA (AGENT)
# ==========================
def get_spec_price(crop):
    market_data = {
        "tomato": {
            "crop": "Tomato",
            "min_price": 2000,
            "max_price": 2500,
            "avg_price": 2250
        },
        "potato": {
            "crop": "Potato",
            "min_price": 300,
            "max_price": 400,
            "avg_price": 350
        }
    }
    return market_data.get(crop.lower(), None)

# ==========================
# AGENT CLASS
# ==========================
class SmartCropAgent:

    def __init__(self, model, scaler):
        self.model = model
        self.scaler = scaler

    def decide(self, crop_name, input_data):
        input_scaled = self.scaler.transform([input_data])
        prediction = self.model.predict(input_scaled)[0]

        market_data = get_spec_price(crop_name)

        max_price = market_data["max_price"] if market_data else 0

        if prediction == 2 and max_price > 2000:
            decision = "SELL"
        elif prediction == 1:
            decision = "HOLD"
        else:
            decision = "STORE"

        return {
            "ML_prediction": int(prediction),
            "market_data": market_data,
            "final_decision": decision
        }

# ==========================
# IMAGE PREDICTION FUNCTION
# ==========================
def predict_image_and_decide(img_path, crop_name):

    # --- Load image for CNN ---
    img = image.load_img(img_path, target_size=(128, 128))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) / 255.0

    # --- CNN Prediction ---
    prediction = cnn_model.predict(img_array)
    class_index = np.argmax(prediction)
    confidence = np.max(prediction)
    freshness = class_names[class_index]

    # --- Agent Decision ---
    agent = SmartCropAgent(model, scaler)
    agent_result = agent.decide(crop_name, X_test[0])
    decision = agent_result['final_decision']
```

```
# ==========================
# DISPLAY IMAGE WITH CLEAN UI
# ==========================
img_cv = cv2.imread(img_path)
img_cv = cv2.cvtColor(img_cv, cv2.COLOR_BGR2RGB)
img_cv = cv2.resize(img_cv, (600, 400))

# Overlay box
overlay = img_cv.copy()
cv2.rectangle(overlay, (0, 0), (600, 120), (0, 0, 0), -1)
alpha = 0.6
img_cv = cv2.addWeighted(overlay, alpha, img_cv, 1 - alpha, 0)

# Title
cv2.putText(img_cv, "AI Crop Analysis", (100, 35),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 1)

# Freshness
cv2.putText(img_cv, f"Freshness: {freshness}", (20, 55),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

# Confidence
cv2.putText(img_cv, f"Confidence: {confidence*100:.2f}%", (20, 85),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 1)

# Decision color
color = (0, 255, 0) if decision == "SELL" else \
        (0, 165, 255) if decision == "HOLD" else \
        (0, 0, 255)

cv2.putText(img_cv, f"Decision: {decision}", (20, 115),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

# Show image
plt.figure(figsize=(7, 5))
plt.imshow(img_cv)
plt.axis('off')
plt.show()

# ==========================
# FINAL OUTPUT
# ==========================
print("\n===== FINAL SYSTEM OUTPUT =====")
print(f"Freshness Prediction : {freshness}")
print(f"Confidence           : {confidence*100:.2f}%")
print(f"Agent Decision       : {decision}")
print(f"Market Data          : {agent_result['market_data']}")

# ==========================
# UPLOAD IMAGE
# ==========================
uploaded = files.upload()

for file_name in uploaded.keys():
    predict_image_and_decide(file_name, "tomato")
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evalu
Choose Files  Sample image.jpg
Saving Sample image.jpg to Sample image (3).jpg
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 124ms/step
```

```
===== FINAL SYSTEM OUTPUT =====
Freshness Prediction : Fresh
Confidence           : 67.25%
Agent Decision       : STORE
Market Data          : {'crop': 'Apple', 'min_price': 1000, 'max_price': 2500, 'avg_price': 1250}
```

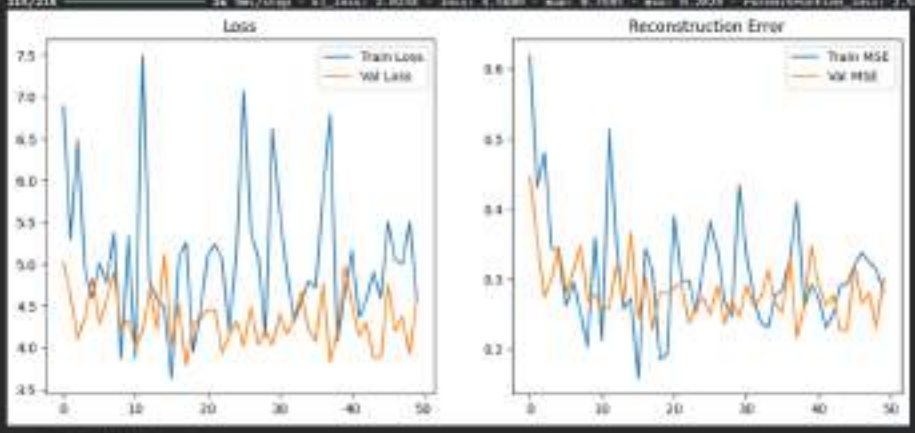
Model Loading

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# ==========================
# PREPARE DATA
# ==========================
X = df[feature_cols].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test = train_test_split(X_scaled, test_size=0.1, random_state=42)

input_dim = X_train.shape[1]

# ==========================
# HYPERPARAMETERS
# ==========================
latent_dim = 8
```

```
learning_rate = 0.001
batch_size = 32

# ============================
# ENCODER
# ============================
class Encoder(tf.keras.Model):
    def __init__(self, latent_dim):
        super().__init__()
        self.dense = layers.Dense(64, activation='relu')
        self.z_mean = layers.Dense(latent_dim)
        self.z_log_var = layers.Dense(latent_dim)

    def call(self, x):
        h = self.dense(x)
        z_mean = self.z_mean(h)
        z_log_var = self.z_log_var(h)

        epsilon = tf.random.normal(shape=tf.shape(z_mean))
        z = z_mean + tf.exp(0.5 * z_log_var) * epsilon

        return z_mean, z_log_var, z


# ============================
# DECODER
# ============================
class Decoder(tf.keras.Model):
    def __init__(self, input_dim):
        super().__init__()
        self.dense = layers.Dense(64, activation='relu')
        self.out = layers.Dense(input_dim)

    def call(self, z):
        h = self.dense(z)
        return self.out(h)


# ============================
# VAE MODEL
# ============================
class VAE(tf.keras.Model):
    def __init__(self, input_dim, latent_dim):
        super().__init__()
        self.encoder = Encoder(latent_dim)
        self.decoder = Decoder(input_dim)

    def call(self, x):
        z_mean, z_log_var, z = self.encoder(x)
        reconstructed = self.decoder(z)
        return z_mean, z_log_var, reconstructed

    def train_step(self, data):
        if isinstance(data, tuple):
            data = data[0]

        with tf.GradientTape() as tape:
            z_mean, z_log_var, reconstruction = self(data)

            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(tf.square(data - reconstruction), axis=1)
            )

            kl_loss = -0.5 * tf.reduce_mean(
                tf.reduce_sum(
                    1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var),
                    axis=1
                )
            )

            total_loss = reconstruction_loss + kl_loss

            # Extra metrics
            mse = tf.reduce_mean(tf.square(data - reconstruction))
            mae = tf.reduce_mean(tf.abs(data - reconstruction))

        grads = tape.gradient(total_loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))

        return {
            "loss": total_loss,
            "reconstruction_loss": reconstruction_loss,
            "kl_loss": kl_loss,
            "mse": mse,
            "mae": mae
        }

    def test_step(self, data):
        if isinstance(data, tuple):
            data = data[0]

        z_mean, z_log_var, reconstruction = self(data)

        reconstruction_loss = tf.reduce_mean(
            tf.reduce_sum(tf.square(data - reconstruction), axis=1)
        )

        kl_loss = -0.5 * tf.reduce_mean(
            tf.reduce_sum(
                1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var),
                axis=1
            )
        )

        total_loss = reconstruction_loss + kl_loss

        mse = tf.reduce_mean(tf.square(data - reconstruction))
        mae = tf.reduce_mean(tf.abs(data - reconstruction))

        return {
            "loss": total_loss,
            "reconstruction_loss": reconstruction_loss,
            "kl_loss": kl_loss,
            "mse": mse,
            "mae": mae
```

```
        )


# ============================
# TRAIN MODEL
# ============================
vae = VAE(input_dim, latent_dim)
vae.compile(optimizer=tf.keras.optimizers.Adam(learning_rate))

history = vae.fit(
    X_train,
    epochs=50,
    batch_size=batch_size,
    validation_data=(X_test, None)
)

# ============================
# PLOT RESULTS
# ============================
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend()
plt.title("Loss")

plt.subplot(1,2,2)
plt.plot(history.history['mse'], label='Train MSE')
plt.plot(history.history['val_mse'], label='Val MSE')
plt.legend()
plt.title("Reconstruction Error")

plt.show()


# ============================
# GENERATE SYNTHETIC DATA
# ============================
num_samples = 5

z_sample = np.random.normal(size=(num_samples, latent_dim))
generated_data = vae.decoder(z_sample).numpy()

generated_data = scaler.inverse_transform(generated_data)

print("\nGenerated Synthetic Crop Data:")
print(generated_data)
```

```
Epoch 46/50
215/215 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - kl_loss: 1.9525 - loss: 4.5925 - mse: 0.3028 - mse: 0.2913 - reconstruction_loss: 2.6398 - val_kl_loss: 1
Epoch 46/50
215/215 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - kl_loss: 2.6341 - loss: 5.5862 - mse: 0.3679 - mse: 0.3388 - reconstruction_loss: 2.8871 - val_kl_loss: 1
Epoch 47/50
215/215 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - kl_loss: 2.0335 - loss: 5.0557 - mse: 0.4053 - mse: 0.3574 - reconstruction_loss: 3.0962 - val_kl_loss: 1
Epoch 48/50
215/215 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - kl_loss: 2.9850 - loss: 5.0861 - mse: 0.3396 - mse: 0.3243 - reconstruction_loss: 2.9586 - val_kl_loss: 1
Epoch 49/50
215/215 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - kl_loss: 2.6364 - loss: 5.5030 - mse: 0.3718 - mse: 0.3141 - reconstruction_loss: 3.0346 - val_kl_loss: 1
Epoch 50/50
215/215 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - kl_loss: 2.8238 - loss: 4.5689 - mse: 0.3585 - mse: 0.2829 - reconstruction_loss: 2.5853 - val_kl_loss: 1
```




```
Generated Synthetic Crop Data:
[[ 2.5530674e+03  2.6213716e+03  1.4035936e+01  1.6778677e+01
   3.1660616e+02  5.8930031e+01  9.8795686e+01  6.9561132e+00
   1.1294814e-04]
 [ 2.3324507e+03  2.6355208e+03  1.4036966e+01  1.1087188e+01
   3.1312658e+02  7.6533608e+01  1.9895653e+02  6.2796418e+00
   1.1895728e-04]
 [ 2.8351365e+03  3.2604701e+03  1.5084886e+01  1.9136172e+01
   4.0234233e+02  8.8206216e+01  7.5376376e+01  7.0473403e+00
   1.4513833e-05]
 [ 3.5180637e+03  4.1657998e+03  1.8018815e+01  1.6177081e+01
   5.5417371e+02  1.0771325e+02  1.5780413e+02  6.9355365e+00
   2.8436721e-01]
 [ 4.5796630e+03  5.2824198e+03  1.5219585e+01  1.4588886e+01
   3.8410816e+02  6.8303145e+01  1.3085718e+02  6.7563246e+00
   7.1986854e-04]]
```

```
!pip install selenium webdriver-manager
```

```
Collecting selenium
  Downloading selenium-4.40.0-py3-none-any.whl.metadata (7.8 kB)
Collecting webdriver-manager
  Downloading webdriver_manager-4.0.2-py2.py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: certifi>=2026.1.4 in /usr/local/lib/python3.12/dist-packages (from selenium) (2026.2.25)
Collecting trio~=0.31.0 (from selenium)
  Downloading trio-0.33.0-py3-none-any.whl.metadata (8.5 kB)
Collecting trio-websocket~=0.12.2 (from selenium)
  Downloading trio_websocket-0.12.2-py3-none-any.whl.metadata (5.1 kB)
Requirement already satisfied: typing_extensions~=5.0,>=4.15.0 in /usr/local/lib/python3.12/dist-packages (from selenium) (4.15.0)
```

```
  Downloading urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Requirement already satisfied: websocket-client~=1.8,>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from selenium) (1.9.0)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from webdriver-manager) (2.32.4)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.12/dist-packages (from webdriver-manager) (1.2.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from webdriver-manager) (26.0)
Requirement already satisfied: attrs>=23.2.0 in /usr/local/lib/python3.12/dist-packages (from trio~=0.31.0->selenium) (26.1.0)
Collecting sortedcontainers (from trio~=0.31.0->selenium)
  Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from trio~=0.31.0->selenium) (3.11)
Collecting outcome (from trio~=0.31.0->selenium)
  Downloading outcome-1.3.0.post0-py2.py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: sniffio>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from trio~=0.31.0->selenium) (1.3.1)
Collecting wsproto~=0.14 (from trio-websocket~=0.12.2->selenium)
  Downloading wsproto-1.3.2-py3-none-any.whl.metadata (5.1 kB)
Requirement already satisfied: pysocks!=1.5.7,<2.0,>=1.5.6 in /usr/local/lib/python3.12/dist-packages (from urllib3[socks]<3.0,>=2.6.3->selenium) (1.
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->webdriver-manager) (3.4.7)
Requirement already satisfied: h11<1,>=0.16.0 in /usr/local/lib/python3.12/dist-packages (from wsproto~=0.14->trio-websocket~=0.12.2->selenium) (
Downloading selenium-4.43.0-py3-none-any.whl (9.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.6/9.6 MB 57.4 MB/s eta 0:00:00
Downloading webdriver_manager-4.0.2-py2.py3-none-any.whl (27 kB)
Downloading trio-0.31.0-py3-none-any.whl (512 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 512.0/512.1 kB 31.8 MB/s eta 0:00:00
Downloading trio_websocket-0.12.2-py3-none-any.whl (21 kB)
Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 131.6/131.6 kB 11.8 MB/s eta 0:00:00
Downloading outcome-1.3.0.post0-py2.py3-none-any.whl (10 kB)
Downloading wsproto-1.3.2-py3-none-any.whl (24 kB)
Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl (29 kB)
Installing collected packages: sortedcontainers, wsproto, urllib3, outcome, trio, webdriver-manager, trio-websocket, selenium
  Attempting uninstall: urllib3
    Found existing installation: urllib3 2.5.0
    Uninstalling urllib3-2.5.0:
      Successfully uninstalled urllib3-2.5.0
Successfully installed outcome-1.3.0.post0 selenium-4.43.0 sortedcontainers-2.4.0 trio-0.31.0 trio-websocket-0.12.2 urllib3-2.6.3 webdriver-manager-4.
```

[1]

```
!apt-get update
!apt-get install -y chromium-browser chromium-chromedriver
```

```
Hit:1 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
Hit:2 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:3 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Hit:4 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:5 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:7 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Hit:8 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:9 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to pro
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
chromium-browser is already the newest version (1:85.0.4183.83-0ubuntu2.22.04.1).
chromium-chromedriver is already the newest version (1:85.0.4183.83-0ubuntu2.22.04.1).
0 upgraded, 0 newly installed, 0 to remove and 38 not upgraded.
```

[2]

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

def get_spot_price(crop_name):
    options = Options()
    options.binary_location = "/usr/bin/chromium-browser"  # 🔥 IMPORTANT

    options.add_argument("--headless")
    options.add_argument("--no-sandbox")
    options.add_argument("--disable-dev-shm-usage")

    driver = webdriver.Chrome(options=options)

    driver.get("https://www.mandi.org/bajarbhav/view-daily-bajarbhav/org/2026-04-14")

    import time
    time.sleep(5)

    rows = driver.find_elements(By.TAG_NAME, "tr")

    for row in rows:
        cols = row.find_elements(By.TAG_NAME, "td")

        if len(cols) >= 5:
            name = cols[0].text.strip().lower()

            if crop_name.lower() in name:
                result = {
                    "crop": name,
                    "min_price": cols[1].text,
                    "max_price": cols[2].text,
                    "avg_price": cols[4].text
                }

                driver.quit()
                return result

    driver.quit()
    return None
```

[3]

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np

# ----------------------------
# LOAD DATA
# ----------------------------
df = pd.read_csv("/content/sample_data/crop_prices.csv")

# Handle missing
df = df.fillna(method='ffill')

# Select features
feature_cols = df.select_dtypes(include=np.number).columns.tolist()
```

```
# Target: create price category
price_col = feature_cols[0]

quantiles = df[price_col].quantile([0.33, 0.66])

df['price_category'] = pd.cut(
    df[price_col],
    bins=[-np.inf, quantiles.iloc[0], quantiles.iloc[1], np.inf],
    labels=[0, 1, 2]
)

X = df[feature_cols].values
y = df['price_category'].astype(int)

# Scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42
)

# ==========================
# TRAIN MODEL (IMPORTANT)
# ==========================
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier()
```

```
def get_spec_price(crop):
    market_data = {
        "tomato": {
            "crop": "টমেটো",
            "min_price": 2000,
            "max_price": 2500,
            "avg_price": 2250
        },
        "potato": {
            "crop": "আলু",
            "min_price": 3000,
            "max_price": 3500,
            "avg_price": 3300
        },
        "cabbage": {
            "crop": "বাঁধাকপি",
            "min_price": 300,
            "max_price": 600,
            "avg_price": 450
        }
    }

    return market_data.get(crop.lower(), None)
```

Web scraping function

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
import time

def get_spec_price(crop_name):
    options = Options()
    options.add_argument("--headless")
    options.add_argument("--no-sandbox")
    options.add_argument("--disable-dev-shm-usage")

    driver = webdriver.Chrome(options=options)

    url = "https://agmarknet.org/bajarbhav/live-daily-bajarbhav/en/2025-01-07"
    driver.get(url)

    time.sleep(5)

    rows = driver.find_elements(By.TAG_NAME, "tr")

    for row in rows:
        cols = row.find_elements(By.TAG_NAME, "td")

        if len(cols) >= 5:
            name = cols[0].text.strip().lower()

            print("Checking:", name)

            if crop_name.lower() in name:
                result = {
                    "crop": name,
                    "min_price": cols[2].text,
                    "max_price": cols[3].text,  # ⚠ FIXED index
                    "avg_price": cols[4].text
                }

                driver.quit()
                return result

    driver.quit()
    return None
```

Agent (ML + Web combined)

```
class SmartCropAgent:

    def __init__(self, model, scaler):
        self.model = model
        self.scaler = scaler

    def decide(self, crop_name, input_data):
```

```
        input_scaled = self.scaler.transform([input_data])
        prediction = self.model.predict(input_scaled)[0]

        market_data = get_open_price(crop_name)

        if market_data:
            max_price = market_data["max_price"]
        else:
            max_price = 0

        if prediction == 2 and max_price > 2000:
            decision = "SELL NOW (High AI + Market)"
        elif prediction == 1:
            decision = "HOLD"
        else:
            decision = "STORE"

        return {
            "AI_prediction": prediction,
            "market_data": market_data,
            "final_decision": decision
        }
```

```
agent = SmartCropAgent(model, scaler)

result = agent.decide("Tomato", X_test[0])

print(result)
```

```
{'AI_prediction': np.int64(0), 'market_data': {'crop': 'ᱴᱚᱢᱟᱴᱚ ᱨᱚᱠᱚᱞ ᱞᱤ', 'min_price': 2000, 'max_price': 2500, 'avg_price': 2250}, 'final_decision': 'STO
```

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import load_model

# ==========================
# LOAD THE MODEL (Fresh/Rotten)
# ==========================
cnn_model = load_model("fresh_rotten_model.keras")

class_names = ['Fresh', 'Rotten'] # adjust if needed

# ==========================
# VIRTUAL MARKET DATA (AGENT)
# ==========================
def get_open_price(crop):
    market_data = {
        "tomato": {
            "crop": "ᱴᱚᱢᱟᱴᱚ ᱨᱚᱠᱚᱞ ᱞᱤ",
            "min_price": 2000,
            "max_price": 2500,
            "avg_price": 2250
        },
        "onion": {
            "crop": "ᱱᱩᱞᱤᱭᱟᱹᱜ",
            "min_price": 300,
            "max_price": 800,
            "avg_price": 750
        }
    }
    return market_data.get(crop.lower(), None)

# ==========================
# AGENT CLASS
# ==========================
class SmartCropAgent:

    def __init__(self, model, scaler):
        self.model = model
        self.scaler = scaler

    def decide(self, crop_name, input_data):

        input_scaled = self.scaler.transform([input_data])
        prediction = self.model.predict(input_scaled)[0]

        market_data = get_open_price(crop_name)

        if market_data:
            max_price = market_data["max_price"]
        else:
            max_price = 0

        if prediction == 2 and max_price > 2000:
            decision = "SELL"
        elif prediction == 1:
            decision = "HOLD"
        else:
            decision = "STORE"

        return {
            "AI_prediction": int(prediction),
            "market_data": market_data,
            "final_decision": decision
        }

# ==========================
# IMAGE PREDICTION FUNCTION
# ==========================
def predict_image_and_decide(img_path, crop_name):

    # --- Image Prediction ---
    img = image.load_img(img_path, target_size=(128, 128))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) / 255.0

    prediction = cnn_model.predict(img_array)
    class_index = np.argmax(prediction)
    confidence = np.max(prediction)

    freshness = class_names[class_index]
```

```
    # --- Agent Decision ---
    agent = SmartCropAgent(model, scaler)
    agent_result = agent.decide(crop_name, X_test[0])

    # --- Display ---
    plt.imshow(img)
    plt.title(f"{freshness} {confidence*100:.2f}%")
    plt.axis('off')
    plt.show()

    print("\n------ FINAL SYSTEM OUTPUT ------")
    print(f"Freshness Prediction: {freshness}")
    print(f"Confidence: {confidence*100:.2f}%")
    print(f"Agent Decision: {agent_result['final_decision']}")
    print(f"Market Value: {agent_result['market_value']}")

# ==============================
# UPLOAD IMAGE (COLAB)
# ==============================
from google.colab import files
uploaded = files.upload()

for file_name in uploaded.keys():
    predict_image_and_decide(file_name, "tomato")
```

```
# ==========================================
# FINAL EXPORT CELL (RUN THIS LAST)
# ==========================================

import joblib
import os
import shutil

# 1. Create export folder
EXPORT_DIR = "/content/exported_models"
os.makedirs(EXPORT_DIR, exist_ok=True)

# 2. Save model + scaler (CRITICAL)
joblib.dump(model, os.path.join(EXPORT_DIR, "crop_model.pkl"))
joblib.dump(scaler, os.path.join(EXPORT_DIR, "scaler.pkl"))

# 3. (Optional but SMART) Save feature columns for consistency
try:
    joblib.dump(feature_cols, os.path.join(EXPORT_DIR, "features.pkl"))
except:
    print("feature_cols not found, skipping...")

# 4. Zip everything (easy download)
zip_path = "/content/crop_model_package.zip"
shutil.make_archive(zip_path.replace(".zip", ""), "zip", EXPORT_DIR)

# 5. Download
from google.colab import files
files.download(zip_path)

print("✅ Model, scaler, and metadata exported successfully!")
```

```
✅ Model, scaler, and metadata exported successfully!
```

Colab paid products - Cancel contracts here

Variables | Terminal | 12:35 PM | Python 3